hSPICE: State-Aware Event Shedding in Complex Event Processing

Ahmad Slo, Sukanya Bhowmik, Kurt Rothermel
University of Stuttgart
firstName.lastName@ipvs.uni-stuttgart.de

ABSTRACT

In complex event processing (CEP), load shedding is performed to maintain a given latency bound during overload situations when there is a limitation on resources. However, shedding load implies degradation in the quality of results (QoR). Therefore, it is crucial to perform load shedding in a way that has the lowest impact on QoR. Researchers, in the CEP domain, propose to drop either events or partial matches (PMs) in overload cases. They assign utilities to events or PMs by considering either the importance of events or the importance of PMs but not both together. In this paper, we propose a load shedding approach for CEP systems that combines these approaches by assigning a utility to an event by considering both the event importance and the importance of PMs. We adopt a probabilistic model that uses the type and position of an event in a window and the state of a PM to assign a utility to an event corresponding to each PM. We, also, propose an approach to predict a utility threshold that is used to drop the required amount of events to maintain a given latency bound. By extensive evaluations on two real-world datasets and several representative queries, we show that, in the majority of cases, our load shedding approach outperforms state-of-the-art load shedding approaches, w.r.t. QoR.

CCS CONCEPTS

• **Information systems** → *Data streams*; *Stream management*; • **Theory of computation** → *Streaming models.*

KEYWORDS

Complex Event Processing, Stream Processing, Load Shedding, Approximate Computing, latency bound, QoS, QoR.

ACM Reference Format:
Ahmad Slo, Sukanya Bhowmik, Kurt Rothermel. 2020. hSPICE: State-Aware Event Shedding in Complex Event Processing. In *The 14th ACM International Conference on Distributed and Event-based Systems (DEBS '20), July 13–17, 2020, Virtual Event, QC, Canada.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3401025.3401742

1 INTRODUCTION

Complex event processing (CEP) systems are used in many applications to detect patterns in input event streams [3, 11, 23]. The criticality of detected patterns (also called complex events) depends on the application. For example, in fraud detection systems in banks, detected complex events might indicate that a fraudster tries to withdraw money from a victim's account. Naturally, the complex events in this application are critical. On the other hand, in applications like network monitoring, soccer analysis, and transportation [12, 17, 18], the detected complex events might be less critical. As a result, these applications might tolerate imprecise detection or loss of some complex events.

In CEP systems, input events are streamed continuously to CEP operators where the input events (or simply events) are partitioned into windows of events. Events within windows are processed by CEP operators to detect patterns (called pattern matching). For most applications, it is important to detect complex events within a certain latency bound (LB) where the late detected complex events become useless [13, 16]. However, if the rate of input events exceeds the processing capacity of CEP operators, the input events queue up and the detection latency of complex events increases, possibly resulting in violation of the given latency bound. For CEP applications that tolerate imprecise detection of complex events and have limited processing resources, one way to keep the given latency bound is by using load shedding [5, 17, 18, 24]. Load shedding reduces the overload on a CEP operator by either dropping events from the operator's input event stream or by dropping a portion of the operator's internal state. This results in decreasing the number of queued events and in increasing the operator processing rate, hence maintaining the given latency bound.

Of course, load shedding may impact the quality of results (QoR) as it might falsely drop complex events (denoted by false negatives) or/and falsely detect complex events (denoted by false positives). Therefore, it is crucial to shed load with minimum adverse impact on QoR. In [5, 18], the authors propose two *black-box* load shedding approaches for CEP systems where their approaches drop input events that have the lowest utility. The approach in [18] uses event type and position within windows as features to probabilistically learn about the utility of events in windows. In [5], the event utility depends on the frequency of events in patterns and in the input event stream. In [17, 24], the authors propose two *white-box* approaches to perform load shedding in CEP where the focus is on dropping partial matches. A partial match is a detected part of a pattern that could become a complex event if the full pattern is matched. However, the approach in [24] might also drop input events if the given latency bound might be violated. Both approaches depend on the following features to learn about the utility of PMs: the progress/state of the PM in the window and the number of remaining events in the window. These two features are used to predict the completion probability and the processing cost of the PMs and hence the PM utilities.

DEBS '20, July 13–17, 2020, Virtual Event, QC, Canada

ACM ISBN 978-1-4503-8028-7/20/07…$15.00
https://doi.org/10.1145/3401025.3401742

In the *black-box* approach, load shedding is performed in a finer granularity (event granularity), i.e., it drops individual events from windows, in comparison to *white-box* dropping approaches which mainly drop PMs, i.e, dropping in a coarser granularity. As a result, the white-box approaches might drop PMs that have relatively high utilities which adversely impacts QoR even if there exist events that may be dropped without impacting QoR. On the other hand, the black-box approaches neither consider the importance nor the state of PMs. An event might have different utilities for individual PMs, depending on the importance and the state of PMs. Thus, in this paper, we propose a new white-box load shedding strategy called hSPICE that combines the best of both black-box and white-box approaches.

In particular, hSPICE is a white-box load shedding approach that drops events from PMs– it sheds on the event-granularity– while considering the operator's internal state. hSPICE predicts the utility of the events using a probabilistic model. The model uses the event type, the event position within a window, and the state of partial matches in a window to learn about the utility of events within windows. An important factor that influences the effectiveness of a load shedding approach is its overhead in performing the load shedding. A high load shedding overhead implies that a high percentage of the available processing power will be used to take the shedding decision. This results in reducing the available processing power to perform pattern matching, thus adversely impacting QoR. As we will show, hSPICE is a lightweight, efficient load shedding approach.

More specifically, our contributions in this paper are as follows:

- We propose a white-box load shedding approach for complex event processing called hSPICE. hSPICE performs load shedding by dropping events from PMs. hSPICE uses a probabilistic model to learn the utility of an event *for a PM* within a window. As learning features, we use the type and position of the event within the window and the state of the PM.
- We provide an algorithm to estimate the number of events to drop to maintain the given latency bound. Additionally, we propose an approach that enables hSPICE to perform load shedding in a lightweight manner.
- We provide extensive evaluations on two real-world datasets and a representative set of CEP queries to prove the effectiveness of hSPICE and to show its performance, w.r.t. its adverse impact on QoR, in comparison to state-of-the-art load shedding approaches.

2 PRELIMINARIES AND PROBLEM STATEMENT

2.1 Complex Event Processing

A CEP system consists of a set of operators that are connected in the form of a directed acyclic graph (DAG). An operator in a CEP system correlates input events to detect patterns. The detected patterns are called complex events. An event in the input event stream (denoted by S_{in}) consists of meta-data and attribute-value pairs. The meta-data contains event type, sequence number and/or timestamp, while the attribute-value pairs represent the event data. For example, the type (denoted by T_e) of event e might represent a company name in a stock application, a player ID in a soccer application, or a bus ID in a transportation application. The event data might contain stock quotes, player positions, or bus locations in these applications. Events in the input event streams have global order, for example, by using the sequence number or the timestamp and a tie-breaker.

Our focus in this paper is on CEP systems consisting of a single operator, where the operator matches one or more patterns (i.e., multi-query). We define the set of patterns that the operator matches as $\mathbb{Q} = \{q_i : 1 \leq i \leq n\}$, where n is the number of patterns. Since patterns might have different importances, each pattern has a weight reflecting its importance. The patterns' weights are determined by a domain expert and they are defined as follows: $\mathbb{W}_{\mathbb{Q}} = \{w_{q_i} : 1 \leq i \leq n\}$, where w_{q_i} is the weight of pattern q_i. In CEP systems, the input event stream S_{in} is continuous and infinite, where the input event stream is partitioned into windows of events. Windows in CEP are opened depending on predicates such as time-based, count-based, or logical predicates. Moreover, the length of windows might be defined by time, event count, or logical conditions [3, 20]. The number of events in a window is defined as window size (denoted by ws). Each event in window w has a position where the position P_e of event e represents the number of events that precedes event e in window w. Windows might overlap which means that there may exist more than one open window at the same time. Hence, event e might belong to multiple windows, where it has different positions P_e within different windows. To clarify the system model, let us introduce the following example.

Example 1. In a stock application, an operator matches pattern q which correlates stock events from three companies. Pattern q is defined as follows: generate a complex event if a change in the stock quote of company A results in a change in the stock quote of company B, followed by a change in the stock quote of company C. We may write this pattern as a sequence operator [4]: $q = seq(A; B; C)$. Hence, the set of patterns that the operator matches is $Q = \{q\}$. In this example, the event type T_e might represent the company name, i.e., A, B, and C. Assume that a count-based predicate is used to open windows where a window is opened every two events, i.e., window slide size is two. Figure 1 depicts this example. Figure 1(a) shows that events in the input event stream (S_{in}) are ordered by the sequence number. Moreover, it shows that there are three open windows which overlap. As an example to show how the same event may have different positions within different windows, we see that the event A_4 from the input event stream belongs to all three windows, where it has the positions 4, 2, and 0 within windows w_1, w_2, and w_3, respectively.

Windows of events are first pushed to the input queue of a CEP operator. The operator continuously gets events from the input queue where, within every window to which an event belongs, the operator checks if the event matches the given pattern(s). We refer to this checking as processing the event within the window. As mentioned above, windows might overlap. However, events within each window are processed independently. A pattern in CEP is modeled as a finite state machine [11, 14] (cf. Figure 1(a)). The set of all possible states $\mathbb{S}_{q_i}$ of pattern $q_i \in \mathbb{Q}$ is defined as: $\mathbb{S}_{q_i} = \{s_k : j \leq k < j + m_i\}$, where m_i represents the number of all possible states of pattern q_i and j represents the sum of the number of all possible states of all patterns $q_l \in \mathbb{Q}$ where $l < i$, i.e.,

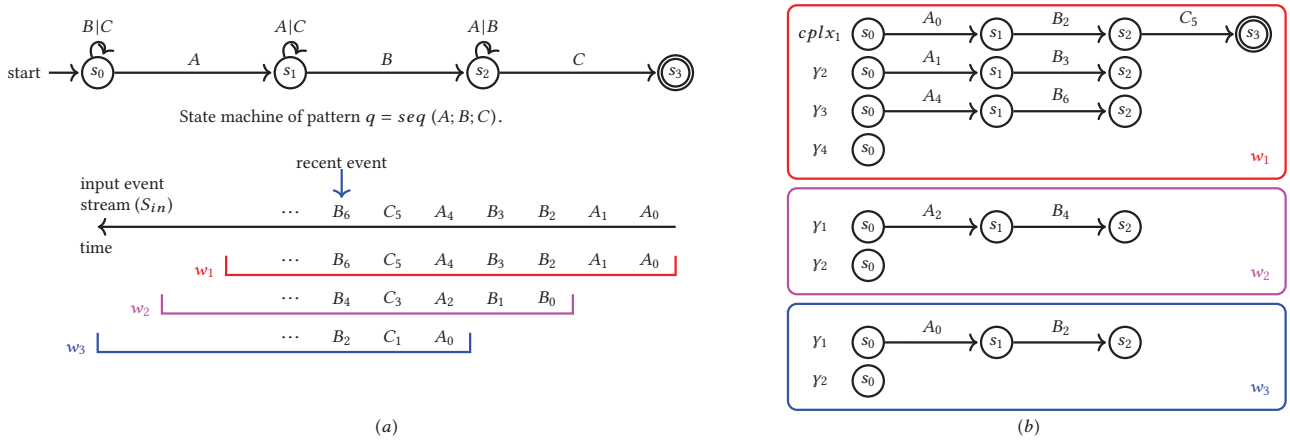


Figure 1: Example 1.

$j = \sum_{l=1}^{i-1} m_l$. In Example 1, pattern q has four states (i.e., $m_i = 4$) where $\mathbb{S}_q = \{s_0, s_1, s_2, s_3\}$ as shown in Figure 1(a). In the figure, s_0 represents the initial state of pattern q and s_3 represents its final state. We define the set of all possible states for all patterns as follows: $\mathbb{S}_{\mathbb{Q}} = \bigcup_{i=1}^{n} \mathbb{S}_{q_i}$. In Example 1, since there is only one pattern (i.e., $\mathbb{Q} = \{q\}$), $\mathbb{S}_{\mathbb{Q}} = \mathbb{S}_q = \{s_0, s_1, s_2, s_3\}$.

Whenever an operator starts to process events within a window, it starts an instance of the state machine of every pattern $q_i \in \mathbb{Q}$ at the initial state. During event processing within a window, an event is matched with the state machine instances of pattern $q_i \in \mathbb{Q}$. The event might cause the state machine instance(s) of pattern q_i to transit between different states of $\mathbb{S}_{q_i}$. Please recall that we have already defined a partial match. However, let us define it more formally. An instance of the state machine of pattern q_i is called a partial match (short PM), where the partial match completes and becomes a complex event if the state machine instance transits to the final state. Hence, processing an event within a window implies that the event is matched with PMs within the window. We define a partial match γ of pattern q_i as $\gamma \subset q_i$. Moreover, we refer to matching event e with PM $\gamma \in q_i$ as processing event e with PM γ, denoted by $e \otimes \gamma$. In Example 1, assume that the operator matches the events in windows chronologically [4] and the operator has already processed all available events in all open windows, i.e., the operator has processed the last event of type B (B_6 in the input event stream) in all windows. Figure 1(b) shows the result of pattern matching in all windows. In window w_1, the operator has detected one complex event ($cplx_1$) while there are still three open PMs in window w_1: γ_2, γ_3, and γ_4 . Similarly, there are two PMs in windows w_2 and w_3 each: γ_1 and γ_2.

Partial match $\gamma \subset q_i$ might be at any state of pattern q_i except the final state, where PM γ at the final state has already completed and become a complex event. Therefore, the set of all possible states ($\mathbb{S}_\gamma$) of PM γ is defined as follows: $\mathbb{S}_\gamma = \mathbb{S}_{q_i} \setminus \{final\ states\}$. Hence, the set of all possible states $\mathbb{S}_\Gamma$ of all PMs of all patterns is defined as follows: $\mathbb{S}_\Gamma = \bigcup_{i=1}^{n} \mathbb{S}_{\gamma_i} : \gamma_i \subset q_i$. In example 1, for PM $\gamma \subset q$, $\mathbb{S}_\gamma = \{s_0, s_1, s_2\}$ and $\mathbb{S}_\Gamma = \mathbb{S}_\gamma = \{s_0, s_1, s_2\}$, as there is only one pattern in this example. We refer to the current state of PM γ as S_γ. Additionally, we refer to PM γ at state s as γ_s. If processing event e with PM $\gamma \subset q_i$ at state s (i.e., $e \otimes \gamma_s$) causes γ to progress, i.e., e matches q_i and causes the state machine instance to transit, we refer to this as event e contributes to PM γ at state s, denoted by $e \in \gamma_s$. In Example 1, event B_0 in window w_2 has been processed with γ_1 at state s_0 (i.e., $B_0 \otimes \gamma_{1_{s_0}}$) but it did not cause γ_1 to progress. While in the same window w_2, event A_2 has been processed with γ_1 at state s_0 (i.e., $A_2 \otimes \gamma_{1_{s_0}}$) and it caused γ_1 to progress to state s_1. Hence, event A_2 contributes to PM γ_1 at state s_0, i.e., $A_2 \in \gamma_{1_{s_0}}$. In window w, at a certain window position P, there might exist one or more PMs belonging to the same or different patterns $q_i \in \mathbb{Q}$. We denote the set of PMs that are currently active at window position P by $\mathbb{\Gamma}_w^P$. Also, we denote the *total* number of PMs that are opened until the end of window w by $\mathbb{\Gamma}_w^T$. In Example 1 Figure1(b), the set of current PMs in windows w_1, w_2 and w_3 are as follows: $\mathbb{\Gamma}_{w_1}^6 = \{\gamma_2, \gamma_3, \gamma_4\}$, $\mathbb{\Gamma}_{w_2}^4 = \{\gamma_1, \gamma_2\}$, and $\mathbb{\Gamma}_{w_3}^2 = \{\gamma_1, \gamma_2\}$. Please note that in the negation operator [14, 22] if the negated event e' contributes to PM γ (i.e., $e' \in \gamma$), PM γ is abandoned. For ease of presentation, hereafter, we also refer to the abandoned PMs as completed PMs.

2.2 Problem Statement

A CEP operator might have limited resources where, in overload cases, it must perform load shedding by dropping a portion of the input events to avoid violating a given latency bound (LB). However, dropping events might degrade QoR, i.e., resulting in false positives and false negatives. Therefore, the load shedding must be performed in a way that has minimum adverse impact on QoR.

As we mentioned above, an operator might detect multiple patterns $\mathbb{Q}$ and each pattern has its weight (i.e., $\mathbb{W}_{\mathbb{Q}}$). For pattern $q_i \in \mathbb{Q}$, we define the number of false positives as FP_{q_i} and the number of false negatives as FN_{q_i}. The total number of false positives (denoted by $FP_{\mathbb{Q}}$) for all patterns is defined as the sum of the number of false positives for each pattern multiplied by the pattern's weight (cf. Equation 1). Similarly, the total number of false negatives (denoted by $FN_{\mathbb{Q}}$) for all patterns is defined as the sum of the number of false negatives for each pattern multiplied by the

pattern's weight (cf. Equation 2).

$$FP_{\mathbb{Q}} = \sum_{q_i \in \mathbb{Q}} w_{q_i}.FP_{q_i} \quad (1)$$

$$FN_{\mathbb{Q}} = \sum_{q_i \in \mathbb{Q}} w_{q_i}.FN_{q_i} \quad (2)$$

As a result, the impact of load shedding on QoR is measured by the sum of the total number of false positives ($FP_{\mathbb{Q}}$) and the total number of false negatives ($FN_{\mathbb{Q}}$). The objective is to minimize the adverse impact on QoR, i.e., minimize ($FP_{\mathbb{Q}} + FN_{\mathbb{Q}}$), while dropping events such that the given latency bound LB is met. More formally, the objective is defined as follows.

$$\begin{aligned} &minimize && (FP_{\mathbb{Q}} + FN_{\mathbb{Q}}) \\ &\text{s.t.} && l_e \leq LB \quad \forall\, e \in S_{in} \end{aligned} \quad (3)$$

where l_e is the latency of event e that represents the sum of the queuing latency of event e and the time needed to process event e within all windows to which event e belongs.

3 LOAD SHEDDING IN CEP

We extend a CEP operator with our proposed load shedding system (hSPICE) that in overload cases drops a portion of the input events to maintain the given latency bound (LB). In CEP, a load shedding system must perform the following three tasks: 1) deciding when input events must be dropped, 2) computing the time interval and the number of events that must be dropped in every time interval (denoted by drop interval) to maintain LB, and 3) dropping input events that have the lowest adverse impact on QoR. Tasks 1 and 2 have already been well studied in literature [17, 18]. Therefore, our focus in this paper is on task 3, i.e, deciding which events to drop. In the following, we shortly explain how tasks 1 and 2 might be performed. Figure 2 depicts a CEP operator extended with two components to enable load shedding: 1) overload detector and 2) load shedder (LS).

The given latency bound (LB), the rate of incoming input events, and the operator throughput (maximum service rate) can be used as parameters to decide when to drop events. The overload detector periodically monitors these parameters. If the input event rate (R) is higher than the operator throughput (μ) for a long enough period, the given latency bound (LB) might be violated. To prevent violating LB, the overload detector requests the load shedder to drop a certain amount of input events. As a drop interval (λ), we might use the window size ws or a part of it as proposed in [18]. Our approach works with any drop interval. However, in this paper, to simplify the presentation, we consider that the drop interval equals the window size, i.e., $\lambda = ws$. The number of events that must be dropped in every window to maintain LB can be computed depending on the input event rate R and the operator throughput μ, where the overload detector computes the drop amount ρ per window (i.e., per drop interval) as follows: $\rho = (1 - \frac{\mu}{R}).ws$. After that, the overload detector sends a command containing the drop interval λ and the number of events ρ to drop per λ to the load shedder. The load shedder drops ρ events per drop interval λ to maintain LB.

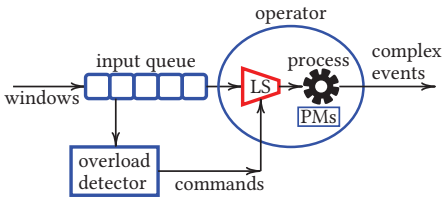


Figure 2: The hSPICE Architecture.

hSPICE

During overload, to maintain the given latency bound (LB), hSPICE drops input events that have the lowest adverse impact on QoR, i.e, on the number of false positives and negatives. To do that, hSPICE assigns utility values to the events where an event that has a high impact on QoR has a high utility and vice versa. On a high abstraction level, hSPICE works as follows. 1) As mentioned above, an event in a window is processed with PMs within the window. Therefore, in a window, hSPICE assigns utility values to an event for each PM within the window individually, i.e., the event gets a certain utility value for each PM within the window. 2) hSPICE performs load shedding by dropping *events* from *partial matches* within windows. In a window, dropping event e from PM γ means that hSPICE prevents processing event e with PM γ within the window.

hSPICE, primarily, performs two tasks: 1) model building and 2) load shedding. In the model building task, hSPICE predicts the event utilities and summarizes the event utilities to reduce the degradation in QoR in overload situations. In the load shedding task, hSPICE drops events to avoid violating the given latency bound. The model building task is not time-critical and can afford to be heavyweight. On the other hand, the load shedding task is time-critical and hence must be lightweight. In the next sections, we describe the above tasks in detail. First, we describe how the utility of an event for a partial match is defined. Then, we explain the way hSPICE predicts the event utility using a probabilistic model. After that, we describe how hSPICE computes the number of events to drop per partial match within windows to maintain the given latency bound. To perform load shedding efficiently, we explain how to predict a utility value that can be used as a threshold utility to drop the required number of events from PMs. Finally, we describe the functionality of the load shedder in hSPICE.

3.1 Event Utility

In a window, only some PMs might complete and become complex events. Hence, PMs in a window might have different importances, w.r.t. QoR. If a PM completes, it is an important PM for QoR. Otherwise, it has no impact on QoR. Moreover, as mentioned above, an event might be processed with one or more PMs within a window, where the event might contribute only to some of these PMs. An event that contributes to a PM might be an important event for the PM since dropping the event from the PM might hinder the PM completion and hence adversely impact QoR. On the other hand, an event that does not contribute to a PM is not important for the PM since dropping the event from the PM does not influence its completion. Therefore, for different PMs in a window, an event might have different importances. As a result, in a window, for

event e and PM γ within the window, hSPICE assigns a utility value to event e (denoted by the utility of event e for PM γ) depending on the importance of PM γ in the window and on the importance of event e for γ. Higher is the importance of γ in the window and higher is the importance of event e for γ, higher is the utility of event e for γ.

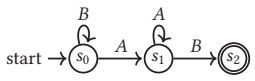
The utility of event e for PM γ of pattern $q_i \in \mathbb{Q}$ within a window (denoted by $U_{e,\gamma}$) depends on three factors: 1) contribution probability—the probability that event e contributes to PM γ, i.e., $e \in \gamma$, 2) completion probability—the probability that PM γ completes, and 3) pattern weight w_{q_i} (given by a domain expert). Clearly, if event e has a high probability to contribute to PM γ, event e is an important event for PM γ. We consider the completion probability of a PM in computing the event utility as well since the PM is only useful if it completes. Therefore, if event e has a high probability to contribute to PM γ and γ has a high probability to complete, event e is an important event and should be assigned a high utility value. This is because dropping event e may hinder PM γ to complete and hence it may adversely impact QoR.

As a result, the utility $U_{e,\gamma}$ of event e for PM $\gamma \subset q_i$ within a window depends on the pattern weight w_{q_i} and the following probability: $P(e \in \gamma \cap \gamma\ completes)$, i.e., the probability that PM γ completes and event e contributes to PM γ. In window w, to predict $P(e \in \gamma \cap \gamma\ completes)$ and hence $U_{e,\gamma}$, hSPICE uses three features: 1) current state S_γ of PM γ, 2) event type T_e, and 3) position P_e of event e in window w. Therefore, the utility $U_{e,\gamma}$ of event e for PM γ of pattern q_i (i.e., $\gamma \subset q_i$) is defined as a function (called utility function) of these three features as shown in Equation 4:

$$U_{e,\gamma} = f(T_e, P_e, S_\gamma) = w_{q_i}.P(e \in \gamma \cap \gamma\ completes) \quad (4)$$

The current state S_γ of PM γ determines which event type(s) enables PM γ to progress, i.e., to transit to a new state(s). Therefore, those two features, i.e., current state S_γ of the PM and event type T_e are important features for computing $U_{e,\gamma}$. For instance, in Example 1, PM γ at state s_0 (i.e., γ_{s_0}), might transit to state s_1 only if event e of type $T_e = A$ is processed with PM γ (i.e., $e \otimes \gamma_{s_0}$).

The position P_e of event e in window w is an important feature to compute $U_{e,\gamma}$ as well since it determines the number of remaining events in the window. If there are still many events remaining in a window, the probability of a PM to complete might be higher than the case where there are only a few remaining events in the window. This is because, in case of many remaining events in a window, a PM has a chance to be processed with more events than in case of only a few remaining events in the window and hence the PM has a higher chance to progress. Moreover, the event position P_e represents the temporal distance between events within the same window. It determines which event instance(s) of the same event type has a higher probability to contribute to a PM in the window as shown in [18]. This is because there exists a correlation between events of certain types at certain positions within a window. A change in an event of a certain type influences the change of events of other types within a certain time interval, i.e., certain position(s) within the window. In Example 1, in window w, a change in the stock quote of company A, i.e., $T_e = A$, at a certain point of time t_1 (i.e., at a certain position in window), might cause a change in the stock quote of company B, i.e., $T_e = B$, within a certain time interval $]t_1, t_2]$, i.e., within certain position(s) in the window.



State machine for pattern $q = seq\ (A; B)$.

T_e/P_e	0	1	2	3	4
A	x	x	x	x	
B			x	x	x

Table 1: Event distribution within windows.

A_0	$ob_e\langle 1-2, s_0, s_1, A_0\rangle : \frac{2}{5}$
A_2	$ob_e\langle 3-4, s_0, s_1, A_2\rangle : \frac{2}{4}$
A_3	$ob_e\langle 5-6, s_0, s_1, A_3\rangle : \frac{2}{2}$

B_3	$ob_e\langle 1, s_1, s_2, B_3\rangle : \frac{1}{4}$
B_4	$ob_e\langle 2-3, s_1, s_2, B_4\rangle : \frac{2}{5}$

$ob_\gamma\langle 1-3,\ completed\rangle$
$ob_\gamma\langle 4-6,\ not\ completed\rangle$

Table 2: Contribution ob_e and completion ob_γ observations.

Figure 3: Observations gathered from six PMs.

s_0					
T_e/P_e	0	1	2	3	4
A	33	0	25	0	0
B	0	0	0	0	0

s_1					
T_e/P_e	0	1	2	3	4
A	0	0	0	0	0
B	0	0	0	25	40

Figure 4: Computing event utility $U_{e,\gamma}$ for a partial match.

3.2 Predicting Event Utility

Having defined the utility $U_{e,\gamma}$ of event e for PM γ, now, we describe how hSPICE predicts the utility $U_{e,\gamma}$ within a window, i.e., $P(e \in \gamma \cap \gamma\ completes)$, hence predicting the value of utility function $f(T_e, P_e, S_\gamma)$ in Equation 4. For ease of presentation, we introduce a simple running example which is depicted in Figures 3 and 4.

Example 2. Let us assume that an operator matches a pattern $q = seq\ (A; B)$, where $\mathbb{S}_q = \{s_0, s_1, s_2\}$ and $\mathbb{S}_\gamma = \{s_0, s_1\}$, $\gamma \subset q$. The used window length is 5 events (i.e., $ws = 5$) and there are only two event types in the input event stream: A and B.

To predict the utility $U_{e,\gamma}$ of event e for PM γ of pattern q_i in window w, we first need to predict the *completion probability* of PM γ, i.e., find the probability that PM γ at state S_γ and at position P_e in window w will complete. Additionally, we need to predict the *contribution probability* of event e to PM γ, i.e., the probability that event e of type T_e at position P_e in window w contributes to PM γ ($e \in \gamma$). If the contribution and completion probabilities are high, then the event utility $U_{e,\gamma}$ is high. On the other hand, if the contribution and/or completion probabilities are low, then the event utility $U_{e,\gamma}$ is low. hSPICE uses statistics gathered over already processed windows to predict the completion and contribution probabilities, thus predicting the event utility for PMs. Next, we first show which statistics hSPICE gathers. Then, we explain the way the event utility $U_{e,\gamma}$ for PMs is predicted depending on those gathered statistics.

Statistic Gathering. To predict the contribution and completion probabilities (i.e., to predict $P(e \in \gamma \cap \gamma\ completes)$), thus predicting the value of utility function f, hSPICE gathers statistics on the progress of PMs within windows during event processing in an operator. To do that, hSPICE uses two types of *observations*:

1) contribution observation, denoted by ob_e, and 2) completion observation, denoted by ob_γ. In window w, for each event e within w, whenever event e is processed with PM γ at state $s = S_\gamma$ (i.e, $e \otimes \gamma_s$), the operator builds an observation of type contribution $ob_e\langle id, s, s', e\rangle$, where id is the id of PM γ. s' represents the state of PM γ after processing event e. If $s \neq s'$, event e has contributed to PM γ at state s, i.e., $e \in \gamma_s$. Additionally, in window w, if PM γ completes, the operator builds an observation of type completion $ob_\gamma\langle id, completed\rangle$, where again id is the id of PM γ. When window w closes (i.e., all its events are processed), all still open PMs in window w, i.e., $\mathbb{\Gamma}_w^P$, (here P is the last position in w) are considered as *not completed* PMs.

Figure 3 shows an example of gathered observations on six PMs. Table 1 shows the distribution of event types in different positions within a window where a cell with x sign in the table means that the corresponding event type might be present at the corresponding position within a window. Please note that event types might not be present in all positions within a window. In the table, for example, the event type A never comes at position 4 in any window and event type B does not come at positions 0 and 1 in any window. Table 2 shows observations on event e of type T_e at position P_e in a window and PM γ at state s only if e contributes to γ (i.e., $e \in \gamma_s$). For example, in the table, event B_3 of type $T_e = B$ at position $P_e = 3$ within windows has never contributed to PM γ at state s_0. Therefore, there are no observations shown in the table on event B_3 with a PM at state s_0. Clearly, if event e is not present at a certain position within windows, event e can not contribute to any PM at this window position. For example, as shown in Table 1, the event of type B never comes at position 1 within windows. Therefore, there are no observations on the event type B at position 1 within windows with a PM at any state. In Table 2, next to each observation of type contribution ob_e, we show the number of PMs at state s to which an event *contributed* divided by the *total* number of PMs at state s with which an event is processed, i.e., $\frac{|\{e:e\in\gamma_s\}|}{|\{e:e\otimes\gamma_s\}|}$. For example, in the table, $ob_e\langle 3-4, s_0, s_1, A_2\rangle : \frac{2}{4}$ means that the event of type $T_e = A$ at position 2 within windows has been processed with four PMs at state s_0. However, it has contributed only to two PMs, in particular, it has contributed to PMs 3 and 4. The table also shows which PMs have completed. For example, in the table, PMs γ_1, γ_2, and γ_3 have completed while PMs γ_4, γ_5, and γ_6 have not completed.

After gathering statistics from η observations, hSPICE uses these observations to predict the utility $U_{e,\gamma}$ of event e for PM γ within window w, i.e., to predict the utility function f (cf. Equation 4).

Utility Prediction. hSPICE uses the gathered observations of both types (contribution ob_e and completion ob_γ) to predict the probability value $P(e \in \gamma \ \cap \ \gamma\ completes)$, hence predicting $U_{e,\gamma}$. First, from both these observation types, hSPICE computes the utility of event e for the set of all possible states of PM γ (i.e., $\mathbb{S}_\gamma$) as follows:

$$U_{e,s} = \frac{|\{e : e \in \gamma_s \ \&\ \gamma\ completed\}|}{|\{e : e \otimes \gamma_s\}|} \tag{5}$$

where $U_{e,s} = P(e \in \gamma_s \ \cap \ \gamma\ completes)$. For event e of certain type T_e at certain position P_e within window w and for PM γ at certain state s, $U_{e,s}$ is computed as a ratio between the number of times PM γ *completes* and event e *contributes* to PM γ at state s (i.e., $e \in \gamma_s$) and the *total* number of times event e is processed with PM γ at state s (i.e., $e \otimes \gamma_s$).

Figure 4 shows the computed utility values $U_{e,s}$ from the observations shown in Table 2. The values are shown as percentage values. The table shows the utility value of event e of type T_e at position P_e within a window for PMs at states s_0 and s_1. For example, in the table, event $e = A_2$ of type $T_e = A$ at position $P_e = 2$ within a window is processed with four PMs at state s_0 (PMs 3, 4, 5, and 6). However, it has contributed only to two PMs (3 and 4). Moreover, since only PM 3 completed, we account for the contribution of event $e = A_2$ only to PM 3. Therefore, in the table, the utility of event type $T_e = A$ at position $P_e = 2$ within a window for a PM at state s_0 equals to 25%, i.e., $U_{e,s_0} = \frac{1}{4} = 25\%$. The event type $T_e = A$ has never contributed to a PM at state s_1 since only the event type $T_e = B$ may contribute to a PM at state s_1. Therefore, the utility of an event of type $T_e = A$ at any position within a window for a PM at state s_1 is always zero as shown in the table. Similarly, the event type $T_e = B$ never contributes to a PM at state s_0. Hence, the utility of an event of type $T_e = B$ at any position within a window for a PM at state s_0 is always zero.

The utility values for all states of PM γ of pattern $q_i \in \mathbb{Q}$ together multiplied by the pattern weight w_{q_i} represent the predicted utility $U_{e,\gamma}$ of event e for PM $\gamma \subset q_i$, where $U_{e,\gamma_s} = f(T_e, P_e, s) = w_{q_i}.U_{e,s}$. Now, we need to store these predicted utility values $U_{e,\gamma}$ for all patterns (i.e., for $\mathbb{Q}$) so that, during load shedding, hSPICE can retrieve them. To reduce the storage overhead, in case of large window size, we use bins to group event utilities. Within window w, the utility values of event e of type T_e at several consecutive window positions (i.e., bin size bs) for PM γ_s at state s are grouped together by taking the average utility value of this event type T_e over all these positions for PM γ_s. For ease of presentation, we will use the bin of size $bs = 1$ if not otherwise stated. To efficiently retrieve the utility values during load shedding, we store the utilities in a table (called utility table UT) of three dimensions (M x N x K), where M represents the number of different event types, $N = \frac{ws}{bs}$, and K is the number of all possible states of all PMs of all patterns, i.e., $K = |\mathbb{S}_{\mathbb{\Gamma}}|$. Therefore, the storage overhead of the utility table UT is $O(M.N.|\mathbb{S}_{\mathbb{\Gamma}}|)$. Each cell $UT(T_e, P_e, S_\gamma)$ in the utility table stores the utility value $U_{e,\gamma}$ of event e of type T_e at position P_e within a window for PM γ at state S_γ, i.e., $U_{e,\gamma} = f(T_e, P_e, S_\gamma) = UT(T_e, P_e, S_\gamma)$. Hence, to get the utility $U_{e,\gamma}$ of event e for PM γ, hSPICE needs to perform only a single lookup in the utility table UT. This means that the time complexity to get $U_{e,\gamma}$ is $O(1)$ which considerably reduces the overhead of load shedding.

The input event stream might change over time, hence the predicted utilities of events for PMs might become inaccurate. One way to capture the changes in the input event stream and keep the event utility accurate is by periodically gathering statistics and recomputing the utility value $U_{e,\gamma}$.

3.3 Drop Amount

As we mentioned above, to maintain the given latency bound (LB) in an overload situation, we must drop ρ events from every window. However, hSPICE drops events from PMs, not from windows, where an event might be dropped from a PM while it is processed with another PM within the same window. Therefore, we must find a

mapping between the number of events to drop per window (ρ) and the number of events to drop per PM within the window. To do that, let us first define the virtual window.

Virtual Window. The virtual window (vw) of window w is a set which contains triplets (e, s, O) consisting of event e of type T_e at position P_e within w, state $s \in \mathbb{S}_{\mathbb{\Gamma}}$, and the number of occurrences $O > 0$ which represents the number of times event e has been processed with a PM at state s within window w. More formally: $vw = \{(e, s, O) : \forall\ e \in w,\ \forall\ \gamma \in \mathbb{\Gamma}_w^T,\ O = |\{\gamma : e \otimes \gamma_s\}| > 0\}$. The virtual window vw of window w contains information on the number of times event e within window w is processed with each distinct state s of a PM in window w. The virtual window depends on the states of PMs in a window. Therefore, it is only possible to know the exact virtual window of window w when all events in window w are processed, i.e., when the set of all PMs $\mathbb{\Gamma}_w^T$ and their states in window w are known. However, we need to know the virtual window of window w before processing all events in window w since we use the virtual window to decide how many and which events must be dropped from PMs within window w.

Therefore, hSPICE predicts virtual window vw of window w by gathering statistics from the operator on already processed windows, denoted by W_{stat}. As mentioned above, in different windows, event distribution might be different (cf. Table 1). Additionally, the occurrences of PM states at certain window positions might also be different in different windows. Hence, different windows might have different corresponding virtual windows. Therefore, to predict virtual window vw of window w, hSPICE first computes virtual window vw_j for each window w_j in the gathered statistics W_{stat}, where $j = 1, .., |W_{stat}|$. Then, hSPICE combines all triplets (e, s, O) from these virtual windows vw_j to construct the virtual window vw by taking the average value for the number of occurrence O of each triplet, i.e., $vw = \{(e, s, O) : e = e_j, s = s_j, O = O + \frac{O_j}{|W_{stat}|},\ \forall\ (e_j, s_j, O_j) \in vw_j\}$. The size of virtual window vw (denoted by ws_v) is computed as the total number of occurrences of each triplet in vw as follows: $ws_v = \sum_{(e,s,O)\in vw} O$. The *virtual window size* represents the *number of times* events are processed with PMs in a window. Therefore, the average number of times (avg_O) an event is processed with a PM in window w is computed as follows: $avg_O = \frac{ws_v}{ws}$. For example, if every event is processed with two PMs within window w, then the virtual window size ws_v is twice the window size ws (i.e., $ws_v = 2.ws$) and $avg_O = 2$.

Dropping an event from window w implies that the event is dropped from the set of all current PMs $\mathbb{\Gamma}_w^P$ within window w. Therefore, if ρ events must be dropped from window w, it implies that, in total, $\rho_v \approx \rho.avg_O \approx \rho.\frac{ws_v}{ws}$ events must be dropped from all PMs $\mathbb{\Gamma}_w^T$ in window w (from virtual window vw of window w, as a shorthand). Hence, dropping ρ events from a window is similar to dropping ρ_v events from its virtual window. One approach to drop ρ_v events from a virtual window (i.e., ρ_v events in total from all PMs in a window) is to drop events equally (for example, equal percentage) from every PM in the window. However, not all PMs in a window have the same importance/same completion probability. Therefore, the drop amount per PM should take into consideration the importance of PMs in the window which in turn minimizes the adverse impact of dropping on QoR. Please note that it is not possible to get the utility of all events for all PMs in a window and then sort them. After that, drop those ρ_v events from PMs that have the lowest utilities. The reason for this is that the event utilities for PMs in a window are only known after processing all events in the window. This is because the event utilities depend on the current state of PMs ($\mathbb{\Gamma}_w^P$) in the window which is only known after processing the events in the window. Next, we explain how to drop the required number of events (ρ_v) from the virtual window of each window while considering the importance of PMs in the window.

Utility Threshold. The approach is to find a utility value (called utility threshold u_{th}) that is used as a threshold value to drop the needed amount of events from virtual window vw of window w. For each triplet (e, s, O) in virtual window vw, we get the utility value $u = U_{e,\gamma_s} = f(T_e, P_e, s)$ from the utility table UT. As the number of occurrences O in the triplet represents the number of times state s might occur at window position P_e, the number of occurrences O implies that the utility value $u = U_{e,\gamma_s}$ might occur O times in virtual window vw, denoted by the utility occurrences O_u for utility u, i.e, $O_u = O$. We accumulate the number of utility occurrences O_u for all utility values in vw in ascending order, denoted by the accumulative utility occurrences OC_u for the utility u, as follows: $OC_u = \sum_{u' \le u} O'_u$. The accumulative utility occurrences OC_u for utility u means that there exist OC_u events in virtual window vw which have a utility value less or equal to the utility value u.

Therefore, using u as a threshold utility u_{th} enables hSPICE to drop OC_u events from PMs in a window. Hence, to drop ρ_v events from the virtual window, we must find a utility value $u = u_{th}$, where $OC_u = \rho_v$. To efficiently retrieve the utility threshold, we store the accumulative utility occurrences in an array (denoted by utility threshold array (UT_{th})) of the same size as the virtual window size ws_v as follows: $UT_{th}(i) = u$, where $i = 1, .., ws_v$ and $OC_u \ge i$ and $OC_u < OC_{u'}\ \forall\ u < u'$. Therefore, to drop ρ_v events from the virtual window, $u_{th} = UT_{th}(\rho_v)$. Hence, the time complexity to get u_{th} is $O(1)$. Please note that predicting the virtual window and building the utility threshold array are done during the model building task. While during the load shedding, hSPICE performs the following two tasks that have a time complexity of $O(1)$: 1) computing how many events to drop (i.e., ρ_v) per virtual window, and 2) determining what utility threshold (i.e., u_{th}) to use.

3.4 Load Shedding

In the above sections, we showed how to compute the utility of events for PMs within a window and how to predict the utility threshold. Now, we describe how hSPICE performs the load shedding, i.e., deciding whether an event should be dropped from a PM or not. Algorithm 1 clarifies how load shedding is performed.

For each event e within window w, before processing e with PM γ in window w, the operator asks the load shedder (LS) whether to drop event e from PM γ. If the LS returns True, the operator drops event e from PM γ, otherwise, it processes event e with PM γ. If there is no overload on the operator, there is no need to drop events and hence LS returns False which means that the operator can process event e with PM γ (cf. Algorithm 1, lines 2-3). On the other hand, if there is an overload on the operator, LS checks whether the utility $U_{e,\gamma}$ of event e for PM γ is higher than the utility threshold u_{th}. Therefore, the LS first gets the utility $U_{e,\gamma}$ of event e for PM γ from the utility table UT, where $U_{e,\gamma} =$

$f(T_e, P_e, S_\gamma) = UT(T_e, P_e, S_\gamma)$. After that, hSPICE compares the utility value with the utility threshold u_{th}, where it returns True if $U_{e,\gamma} \leq u_{th}$, otherwise hSPICE returns False (cf. Algorithm 1, lines 4-7). This shows that hSPICE is lightweight in performing load shedding where the time complexity to decide whether or not to drop an event from a PM is $O(1)$.

Algorithm 1 Load shedder.

```
1: drop (T_e, P_e, S_γ) begin
2:   if !isOverloaded then ▷ there is no overload hence no need to drop events
3:       return False
4:   else if UT(T_e, P_e, S_γ) ≤ u_th then
5:       return True
6:   else
7:       return False
8: end function
```

4 PERFORMANCE EVALUATIONS

In this section, we evaluate the performance of hSPICE by using two real-world datasets and several representative queries.

4.1 Experimental Setup

Evaluation Platform. We run our evaluations on a machine that is equipped with 8 CPU cores (Intel 1.6 GHz) and a main memory of 24 GB. The OS used is CentOS 6.4. We run a CEP operator in a single thread on this machine, where this single thread is used as a resource limitation. Please note, the resource limitation can be any number of threads/cores and the behavior of hSPICE does not depend on a specific limitation. We implemented hSPICE by extending a prototype CEP framework that is implemented using Java.

Baseline. We compare the performance of hSPICE with three state-of-the-art load shedding strategies: 1) eSPICE: it is a black-box load shedding approach that drops events from windows [18]. 2) BL: we also implemented a black-box load shedding strategy (denoted by BL) similar to the one proposed in [5]. Additionally, it captures the notion of weighted sampling techniques in stream processing [19]. BL drops events from windows, where an event type (e.g., player ID or stock symbol) receives a higher utility proportional to its repetition in patterns and in windows. Then, depending on event type utilities, it uses uniform sampling to decide which event instances to drop from the same event type. 3) pSPICE: it is a white-box load shedding strategy that drops PMs [17].

Datasets. We use two real-world datasets. 1) A stock quote stream from the New York Stock Exchange, which contains real intra-day quotes of different stocks from NYSE collected over two months from Google Finance [2]. 2) A position data stream from a real-time locating system (denoted by RTLS) in a soccer game [1]. Players, balls, and referees are equipped with sensors that generate events containing their position, velocity, etc.

Queries. We apply four queries (Q_1, Q_2, Q_3, and Q_4) that cover an important set of operators in CEP as shown in Table 3: sequence operator, sequence operator with repetition, sequence with negation operator, and sequence with any operator, all with skip-till-next/any-match [3, 4, 22]. Moreover, the queries use time-based sliding window strategy.

	Stock queries
Q_1	**pattern seq**($C_1; C_2; ..; C_{10}$) **where** *all* C_i *rise by x%* **or** *all* C_i *fall by x%*, $i = 1..10$ **within** *ws* minutes
Q_2	**pattern seq**($C_1; C_1; C_2; C_3; C_2; C_4; C_2; C_5; C_6; C_7; C_2; C_8; C_9; C_{10}$) **where** *all* C_i *rise by x%* **or** *all* C_i *fall by x%*, $i = 1..10$ **within** *ws* minutes
Q_3	**pattern seq**($C_1; C_2; C_3; C_4; !C_5; C_6; C_7; C_8; C_9; C_{10}$) **where** *all* C_i *rise by x%* **and** C_5 *does not rise by y%* **or** *all* C_i *fall by x%* **and** C_5 *does not fall by y%* , $i = 1..10$ *and* $i \neq 5$ **within** *ws* minutes
	Soccer queries
Q_4	**pattern seq**(S; **any**($3, D_1, D_2, .., D_n$)) **where** *S possesses ball* **and** *distance*(S, D_i) $\leq x$ *meters* , $i = 1..n$ and n is the number of players in a team **within** *ws* seconds

Table 3: Queries.

In Table 3, we use *ws* to refer to the window length. For stock queries (Q_1, Q_2, Q_3), C_i represents the stock quote of company i. Q_1 detects a complex event when rising or falling stock quotes of 10 certain stock symbols, by a given percentage, are detected within *ws* minutes in a certain sequence. Q2 detects a complex event when 10 rising or 10 falling stock quotes of certain stock symbols *with repetition*, by a given percentage, are detected within *ws* minutes in a certain sequence. Q_3 is similar to Q_1 but it detects a complex event only if the stock quote of a certain company (i.e., C_5) does not change by a given percentage. Q_4 uses the RTLS dataset and it detects a complex event when any 3 defenders of a team (defined as D_i) defend against a striker (defined as S) from the other team within *ws* seconds from the ball possessing event by the striker. The defending action is defined by a certain distance between the striker and the defenders. For this query, we use two strikers, one from each team.

4.2 Experimental Results

In this section, we evaluate the performance of hSPICE in comparison with other load shedding strategies. First, we show its impact on QoR, i.e., the number of false negatives and the number of false positives. Then, we show how good hSPICE is in maintaining the given latency bound (*LB*).

If not stated otherwise, we use the following settings. For all queries Q_1, Q_2, Q_3, and Q_4, we use a *time-based* sliding window and a *time-based* predicate. A new window is opened for Q_1, Q_2, Q_3 every 1 minute, i.e., the slide size is 1 minute. For Q_4, a new window is opened every 1 second. We stream events to the operator from the datasets that are stored in files. We first stream events at input event rates which are less or equal to the operator throughput μ (maximum service rate) until the model is built. After that, we increase the input event rate to enforce load shedding as we will mention in the following experiments. The used latency bound $LB = 1$ second. We configure all load shedding strategies (i.e., hSPICE, eSPICE, BL, and pSPICE) to have a safety bound, where they start dropping events/PMs when the event queuing latency is greater than or equal to 80 % of LB, i.e., the safety bound equals to

200 milliseconds. We execute several runs for each experiment and show the mean value and standard deviation.

An important factor that might influence QoR is the input event rate. Higher is the input event rate, higher is the amount of events that must be dropped and hence higher is the impact of load shedding on QoR. Additionally, other factors that might impact QoR are the query properties, e.g., the used window size. Therefore, next, we show the impact of these factors on QoR, i.e., on false negatives and positives. Please note that applying load shedding might result in false negatives for all queries Q_1, Q_2, Q_3, and Q_4. However, it might result in false positives only in case of Q_3 since Q_3 has a negation operator. If the negated event is dropped by the load shedder, it might result in a false positive.

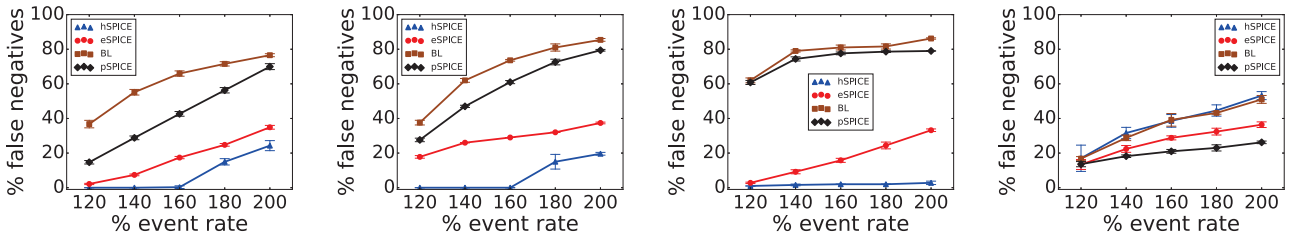
4.2.1 Impact of Event Rate on QoR. To evaluate the performance of hSPICE, we run experiments with queries Q_1, Q_2, Q_3, and Q_4. To show the impact of input event rate, we stream both datasets to the operator with input event rates that are higher than the operator throughput μ by 20%, 40%, 60%, 80%, and 100% (i.e., event rate= 120%, 140%, 160%, 180%, and 200% of the operator throughput μ). Moreover, for Q_1, Q_2, and Q_3, we use the following window sizes, respectively: 18, 35, and 25 minutes. For Q_4, the used window size is 30 seconds. The measured operator throughput μ (without load shedding) for queries Q_1, Q_2, Q_3, and Q_4 are as follows: 23K, 14K, 36K, 27K events/second, respectively.

Impact on False Negatives. Figure 5 depicts the impact of event rates on false negatives for all queries. Figure 6 shows the ratio of dropped events/PMs with different event rates for Q_1 and Q_4. We observed similar results for Q_2 and Q_3, hence we do not show them. In both figures, the x-axis represents the event rate. The y-axis in Figure 5 represents the percentage of false negatives while, in Figure 6, it represents the ratio of dropped events/PMs.

The percentage of false negatives might increase if the input event rate increases since more events/PMs must be dropped. Figure 5a and Figure 6a show the percentage of false negatives and the percentage of drop ratio for Q_1, respectively. As shown in Figure 5a, hSPICE has almost no impact on false negatives when the event rate is less or equal to 160% although hSPICE drops up to 80% of events when the event rate is 160% as depicted in Figure 6a. Increasing the event rate by more than 160% forces hSPICE to produce false negatives where the percentage of false negatives is 15% and 22% using event rates of 180% and 200%, respectively. The drop ratio starts to decrease when using a high event rate as shown in Figure 6a when using the event rate of 200%. The reason behind this is that when more events should be dropped, events with high utilities might be dropped. Dropping events with high utilities might hinder opening new PMs which in turn reduces the number of events that must be dropped. Since hSPICE drops more events compared to other load shedding strategies, i.e., eSPICE and BL, the impact of shedding in hSPICE on opening new PMs is higher which results in decreasing its drop ratio when the event rate is 200%. However, not opening those PMs might increase the number of false negatives. The percentage of false negatives caused by other load shedding strategies also increases when the event rate increases. As depicted in Figure 5a, when the event rate increases from 120% to 200%, the percentage of false negatives for eSPICE, BL, and pSPICE increases from 2% to 35%, from 31% to 77%, and from 15% to 72%, respectively. Moreover, the drop ratio increases with the event rate as shown in Figure 6a. This shows that hSPICE significantly outperforms all other load shedding strategies for Q_1 (sequence operator). The results for Q_2 (sequence operator with repetition) are similar to the results for Q_1 as depicted in Figure 5b where hSPICE also outperforms, w.r.t. the percentage of false negatives, all other load shedding strategies.

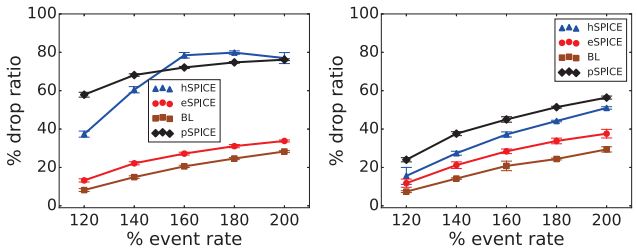
Figure 5c depicts the percentage of false negatives for Q_3 (sequence with negation operator). In Q_3, we limit the number of complex events to only one event per window, where the window is closed if a complex event is detected. We do that to determine the impact of the negation operator on the matching output. The performance of hSPICE, w.r.t. the percentage of false negatives, over Q_3 is considerably better than the performance of hSPICE over Q_1 and Q_2. The reason behind this is that, in Q_3, there is at most one complex event per window in comparison to Q_1 and Q_2 that detect all possible complex events in a window. Hence, in the case of Q_3, there exist many events in the window that have low utilities where dropping those events do not influence the percentage of false negatives. Figure 5c shows that using hSPICE with different event rates introduces almost zero false negatives. The percentage of false negatives caused by using other load shedding strategies increases with increasing event rate. In Figure 5c, the percentage of false negatives produced by eSPICE, BL, and pSPICE increases from 3% to 35%, from 62% to 84%, and from 59% to 79% when increasing the event rate from 120% to 200%, respectively. This shows that, for Q_3, hSPICE drastically reduces the percentage of false negatives compared to the other load shedding strategies.

Figures 5d and 6b show the percentage of false negatives and the percentage of drop ratio for Q_4 (sequence with any operator), respectively. The drop ratio in Figure 6b increases when the event rate increases. However, the drop ratio of hSPICE for Q_4 is lower than its drop ratio for Q_1. This is because the cost of processing events in Q_4 is higher than the cost of processing events in Q_1. Therefore, in Q_4, the overhead of performing load shedding in comparison to the event processing cost is lower which results in a low drop ratio. In Figure 5d, the percentage of false negatives caused by hSPICE increases from 13% to 52% when increasing the event rate from 120% to 200%, respectively. Whereas, the percentage of false negatives caused by eSPICE, BL, and pSPICE increases from 13% to 37%, from 17% to 50%, and from 12% to 26% when increasing the event rate from 120% to 200%, respectively. This shows that hSPICE performs almost worse than all other load shedding strategies. The reason behind this is that the overhead of hSPICE is high in comparison to other load shedding strategies. For every event in a window, hSPICE checks whether to drop the event or not from every individual PM within the window which increases the overhead of performing load shedding in hSPICE. While eSPICE and BL, for example, check whether to drop the event or not from the window regardless of the number of PMs within the window which reduces the overhead of performing load shedding in these approaches. The overhead of hSPICE is high in all queries, however, the impact of hSPICE overhead is worse in Q_4. This is because in Q_4 the utility values are spread and less accurately predicted since Q_4 represents an *any* operator in comparison to other queries that use a *sequence* operator. Q_4 matches an event of any type (any player) with a PM at any state, unlike the *sequence* operator that matches



(a) Q_1 (b) Q_2 (c) Q_3 (d) Q_4

Figure 5: Impact of event rate on false negatives.



(a) Q_1 (b) Q_4

Figure 6: Impact of event rate on drop ratio.

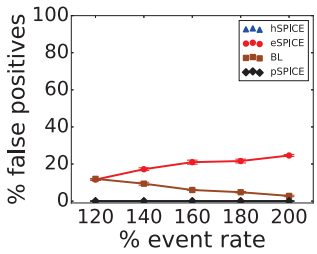


Figure 7: Impact of event rate on false positives.

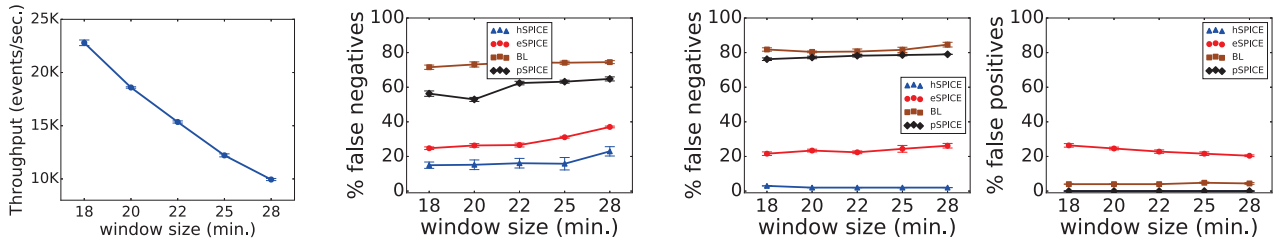
only an event of a certain type with a PM at a certain state. Hence, in the case of Q_4, the majority of events in a window have similar utilities for all PM states.

Impact on False Positives. As we mentioned above, only Q_3 (sequence with negation operator) might have false positives. Therefore, next, we analyze the impact of load shedding on the false positives using Q_3. Figure 7 depicts the percentage of false positives with different event rates when using load shedding over Q_3. In the figure, the x-axis represents the event rate and the y-axis represents the percentage of false positives. Figure 7 shows that hSPICE performs very well with the negation operator where the percentage of false positives is almost zero for different event rates. Please recall that Q_3 detects at most one complex event per window.

In the figure, increasing the event rate results in increasing the percentage of false positives when using eSPICE. The percentage of false positives caused by eSPICE increases from 12% to 24% when increasing the event rate from 120% to 200%. However, the figure shows that the percentage of false positives produced when using BL decreases from 12% to 3% when increasing the event rate from 120% to 200%. The reason behind this is that, for low event rates, BL needs to drop fewer events and hence more redundant events might exist in windows that might match the pattern. On the other hand, with a high event rate, BL must drop more events which makes it hard to have redundant events that might match the pattern. Higher is the probability to match the pattern, higher is the probability to get false positives. pSPICE drops PMs, hence it can not produce any false positive.

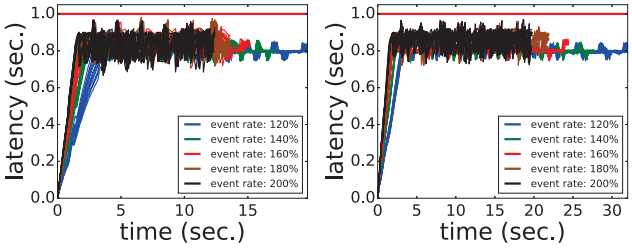
4.2.2 Impact of Window Size on QoR. In this section, we analyze the impact of window size on QoR. To do that, we run experiments with queries Q_1 and Q_3 where we use a fixed event rate of 180%, i.e., the input event rate is higher than the operator throughput μ by 80%. To show the impact of window size, we vary the window size for both Q_1 and Q_3. The used window sizes for Q_1 and Q_3 are as follows: 18, 20, 22, 25, and 28 minutes. Figure 8 depicts the results for both queries. Figure 8a shows the operator throughput μ (without load shedding) for Q_1 with different window sizes. If the window size increases, the number of overlapped windows increases and hence an event becomes a part of more windows. This implies that the operator throughput decreases since events must be processed in more windows. This is observed in Figure 8a where the operator throughput decreases from 23K to 10K when the window size increases from 18 to 28 minutes. The operator throughput for Q_3 has a similar behavior, hence we do not show it.

Figure 8b depicts the percentage of false negatives for Q_1. Increasing the window size might result in increasing the completion probability of PMs within the window. This implies that more events in the window might acquire a high utility value. Therefore, in this case, the load shedding impact on QoR might increase. Moreover, increasing the window size might increase the number of concurrent PMs within the window where more PMs might open. This implies that the overhead of load shedding of hSPICE might increase with increasing window size since the overhead of load shedding in hSPICE is proportional to the number of PMs in windows. This might result in dropping more events hence increasing the impact on QoR. This is observed in Figure 8b where the percentage of false negatives caused by hSPICE increases from 18% to 21% when the window size increases from 18 to 28 minutes. This also happens when using eSPICE where the percentage of false negatives increases from 23% to 38% when increasing the window size from 18 to 28 minutes. The results for pSPICE are also similar. However, the results for BL shows that the window size has almost no influence on the percentage of false negatives. This shows that



(a) Q_1: operator throughput μ (b) Q_1: false negatives (c) Q_3: false negatives (d) Q_3: false positives

Figure 8: Impact of window size on QoR.



(a) Q1 (b) Q2

Figure 9: Maintaining latency bound.

hSPICE outperforms, w.r.t. the percentage of false negatives, all other load shedding strategies regardless of the used window sizes.

Figure 8c shows the percentage of false negatives for Q_3. In the figure, the percentage of false negatives using hSPICE is always negligible. This is because, as we mentioned above, Q_3 matches at most one complex event per window, hence there might exist many events with low utilities where dropping those events has no impact on QoR. In the figure, the percentage of false negatives using eSPICE slightly increases when increasing the window size. The results for BL and pSPICE show that the percentage of false negatives stay almost stable with different window sizes. Again for Q_3, hSPICE outperforms, w.r.t. false negatives, all other load shedding strategies irrespective of the used window sizes. Figure 8d depicts the percentage of false positives for Q_3. The figure shows that the percentage of false positives caused by hSPICE is, again, negligible for different window sizes. In the figure, the percentage of false positives for eSPICE slightly decreases while it stays stable for BL. As mentioned above, pSPICE does not result in false positives.

4.2.3 Maintaining Latency Bound. The main objective of hSPICE is to minimize the degradation in QoR while maintaining a given latency bound (LB). As mentioned above, LB is 1 second and hSPICE drops events when the event queuing latency is greater than or equal to 80% of LB (i.e., 800 milliseconds). The event rate is an important factor that influences the ability of hSPICE to maintain LB. Therefore, next, we show the ability of hSPICE to maintain the given latency bound (LB) with different event rates. To do that, we evaluate hSPICE with all queries using the same setting as in Section 4.2.1. Figure 9 shows the event latency for Q_1 and Q_2 where the event latency is the sum of the event queuing latency and the event processing latency. In the figure, the x-axis represents the event rate and the y-axis represents the induced event latency. We observed similar results for Q_3 and Q_4, hence we do not show them.

Figures 9a and 9b depict results for Q_1 and Q_2, respectively. The figures show that hSPICE always maintains the given latency bound regardless of the event rate. In the figure, the induced event latency stays around 800 milliseconds (i.e., 80% of LB which is used to have a safety bound).

4.2.4 Discussion. hSPICE shows its ability to maintain the given latency bound while minimizing the degradation in QoR. Through extensive evaluations, we show that hSPICE outperforms, w.r.t. QoR, eSPICE, BL, and pSPICE for the majority of queries– especially for *sequence* operators. The performance of hSPICE for the *any* operator is worse than the performance of other load shedding strategies. We also show that increasing the window size might increase the impact of hSPICE on QoR due to the following reasons. The overhead of load shedding might increase and hSPICE might have to drop more events from PMs in the window. Moreover, the PM completion probability might increase when the window size increases, hence more events in a window might become more important/get high utilities.

5 RELATED WORK

Complex event processing (CEP) systems are used in many applications to detect interesting patterns in input event streams [3, 9, 11, 23]. There exist several well-defined patterns in CEP (also called operators), e.g., sequence, negation, any, disjunction, and conjunction [4, 10, 22]. In CEP systems, the input event stream is continuous and may have a high volume. Moreover, the events are usually required to be processed in near real-time [13, 16]. Therefore, in CEP, there exist several techniques aiming to process the input events in a given latency bound such as parallelism, optimizations, and pattern sharing [3, 11, 14, 22]. However, these techniques are not always sufficient or even possible, therefore, researchers propose to use load shedding.

Recently, there have been several works on load shedding in CEP [5, 17, 18, 24]. All these approaches aim to minimize the impact of load shedding on QoR. The approaches in [5, 18] propose to drop events with the lowest utility from a CEP operator while the works in [17, 24] mainly drop PMs with the lowest utility in overload situations. Besides, the authors in [24] propose to drop also events if the given latency bound might be violated. In [5], the utility of an event depends on the event type and its frequency in the input event stream. While in [18] the utility of an event depends on the event type and its position in the window. In [17, 24], the utility of a PM depends on its completion probability and its

estimated processing cost. To predict the utility of a PM, the authors propose to use as learning features the current state of the PM and the remaining events in the window. Unlike all these approaches, our approach drops events from PMs where an event might have different importance for different PMs. As a result, our approach predicts the event utilities more accurately and performs dropping more precisely, thus reducing the adverse impact of load shedding on QoR.

In the domain of approximate CEP, the authors in [8] propose a white-box approach (called RC-ACEP) to drop events from PMs in overload cases. The approach aims to minimize the degradation in QoR. They assign utilities to PMs depending on completion probabilities of the PMs– higher is the completion probability, higher is the utility. The idea is to process input events firstly with PMs that have the highest utilities. For each newly coming input event, RC-ACEP stops processing the previous event, recalculates and sorts PM utilities, and then processes the new events with the sorted PMs. However, recalculating and sorting PM utilities for every input event imposes a high overhead. Moreover, they do not consider the importance of input events for PMs where input events might have different importance for different PMs.

Load shedding is also extensively researched in the stream processing domain [6, 7, 12, 13, 13, 15, 19–21]. In [7, 12, 19], the authors assume that the importance of a tuple depends on the tuple's content. [19] assumes the mapping between the utility and tuple's content is given, for example, by an application expert, while [12, 19] learn this mapping online depending on the used query. The authors in [15] assume that the importance of a tuple depends on the processing latency of the tuple– higher is the processing latency of a tuple, lower is its importance. Therefore, they drop those tuples that have the highest processing latencies. In [13], the authors fairly select tuples to drop from different input streams by combining two techniques: stratified sampling and reservoir sampling. The authors in [21] also propose to use stratified sampling and reservoir sampling to perform the approximate join. In both these papers, the authors assume that tuples have the same utility values and impose the same processing latency. All these works do not capture the correlation between events in patterns which is important in CEP. For example, if the pattern is $seq(A; B)$, then events of type A are only important if the stream contains events of type B and vise-versa. Our approach implicitly captures this correlation.

6 CONCLUSION

In this paper, we proposed an efficient, lightweight load shedding strategy called hSPICE which combines the advantages of both black-box and white-box state-of-the-art load shedding strategies. In overload cases, hSPICE drops events from partial matches to maintain a given latency bound. To assign a utility value to an event for a partial match, hSPICE uses three features: 1) event type, 2) event position in the window, and 3) the current state of the partial match. By using a probabilistic model, hSPICE uses these features to predict the event utility. Through extensive evaluations on two real-world datasets and several representative queries, we show that, for the majority of queries, hSPICE outperforms, w.r.t. QoR, state-of-the-art load shedding strategies. Moreover, we show that hSPICE always maintains the given latency bound regardless of the incoming input event rate.

ACKNOWLEDGEMENT

This work was supported by the German Research Foundation (DFG) under the research grant "PRECEPT II" (BH 154/1-2 and RO 1086/19-2). The authors would like to thank Nabila Hashad for helping with implementation.

REFERENCES

[1] [n.d.]. *DEBS 2013.* https://debs.org/grand-challenges/2013/ Accessed: 2019-08-16.
[2] [n.d.]. Google Finance. https://www.google.com/finance. 05.05.2019.
[3] Cagri Balkesen, Nihal Dindar, Matthias Wetter, and Nesime Tatbul. 2013. RIP: Run-based Intra-query Parallelism for Scalable Complex Event Processing. In *Proc. of the 7th ACM DEBS Conf. on Distributed Event-based Systems.*
[4] S. Chakravarthy and D. Mishra. 1994. Snoop: An Expressive Event Specification Language for Active Databases. *Data Knowl. Eng.* 14, 1 (Nov. 1994), 1–26.
[5] Yeye He, Siddharth Barman, and Jeffrey F. Naughton. 2014. On Load Shedding in Complex Event Processing. In *ICDT.*
[6] Evangelia Kalyvianaki, Marco Fiscato, Theodoros Salonidis, and Peter Pietzuch. 2016. THEMIS: Fairness in Federated Stream Processing Under Overload. In *Proc. of the Int. Conf. on Management of Data.*
[7] N. R. Katsipoulakis, A. Labrinidis, and P. K. Chrysanthis. 2018. Concept-Driven Load Shedding: Reducing Size and Error of Voluminous and Variable Data Streams. In *IEEE Int. Conf. on Big Data.*
[8] Zheng Li and Tingjian Ge. 2016. History is a Mirror to the Future: Best-effort Approximate Complex Event Matching with Insufficient Resources. *Proc. VLDB Endow.* 10, 4 (Nov. 2016), 397–408.
[9] G. F. Lima, A. Slo, S. Bhowmik, M. Endler, and K. Rothermel. 2018. Skipping Unused Events to Speed Up Rollback-Recovery in Distributed Data-Parallel CEP. In *2018 IEEE/ACM 5th International Conference on Big Data Computing Applications and Technologies (BDCAT).* 31–40. https://doi.org/10.1109/BDCAT.2018.00013
[10] M. Liu, M. Li, D. Golovnya, E. A. Rundensteiner, and K. Claypool. 2009. Sequence Pattern Query Processing over Out-of-Order Event Streams. In *IEEE 25th Int. Conf. on Data Engineering.*
[11] Ruben Mayer, Ahmad Slo, Muhammad Adnan Tariq, Kurt Rothermel, Manuel Gräber, and Umakishore Ramachandran. 2017. SPECTRE: Supporting Consumption Policies in Window-based Parallel Complex Event Processing. In *Proc. of the 18th ACM/IFIP/USENIX Middleware Conf.*
[12] Chris Olston, Jing Jiang, and Jennifer Widom. 2003. Adaptive Filters for Continuous Queries over Distributed Data Streams. In *Proc. of the ACM SIGMOD Int. Conf. on Management of Data.*
[13] Do Le Quoc, Ruichuan Chen, Pramod Bhatotia, Christof Fetzer, Volker Hilt, and Thorsten Strufe. 2017. StreamApprox: Approximate Computing for Stream Analytics. In *Proc. of the 18th ACM/IFIP/USENIX Middleware Conf.*
[14] Medhabi Ray, Chuan Lei, and Elke A. Rundensteiner. 2016. Scalable Pattern Sharing on Event Streams. In *Proc. of the Int. Conf. on Management of Data.*
[15] Nicoló Rivetti, Yann Busnel, and Leonardo Querzoni. 2016. Load-aware Shedding in Stream Processing Systems. In *Proc. of the 10th ACM Int. Conf. on Distributed and Event-based Systems.*
[16] Henriette Röger, Sukanya Bhowmik, and Kurt Rothermel. 2019. Combining It All: Cost Minimal and Low-Latency Stream Processing across Distributed Heterogeneous Infrastructures. In *Proceedings of the 20th International Middleware Conference* (Davis, CA, USA) *(Middleware '19).* 13.
[17] Ahmad Slo, Sukanya Bhowmik, Albert Flaig, and Kurt Rothermel. [n.d.]. pSPICE: Partial Match Shedding for Complex Event Processing. In *IEEE BigData 2019.*
[18] Ahmad Slo, Sukanya Bhowmik, and Kurt Rothermel. 2019. eSPICE: Probabilistic Load Shedding from Input Event Streams in Complex Event Processing. In *Proceedings of the 20th International Middleware Conference* (Davis, CA, USA) *(Middleware '19).* ACM, 13.
[19] Nesime Tatbul, Uğur Çetintemel, Stan Zdonik, Mitch Cherniack, and Michael Stonebraker. 2003. Load Shedding in a Data Stream Manager. In *Proc. of the 29th Int. Conf. on Very Large Data Bases.*
[20] Nesime Tatbul and Stan Zdonik. 2006. Window-aware Load Shedding for Aggregation Queries over Data Streams. In *Proc. of the 32nd Int. Conf. on Very Large Data Bases.*
[21] Wee Hyong Tok, Stéphane Bressan, and Mong-Li Lee. 2008. A Stratified Approach to Progressive Approximate Joins. In *Proc. of the Int. Conf. on Extending Database Technology: Advances in Database Technology.*
[22] Eugene Wu, Yanlei Diao, and Shariq Rizvi. 2006. High-performance Complex Event Processing over Streams. In *Proc. of the ACM SIGMOD Int. Conf. on Management of Data.*
[23] N. Zacheilas, V. Kalogeraki, N. Zygouras, N. Panagiotou, and D. Gunopulos. 2015. Elastic complex event processing exploiting prediction. In *IEEE Int. Conf. on Big Data.*
[24] Bo Zhao, Nguyen Quoc Viet Hung, and Matthias Weidlich. [n.d.]. Load Shedding for Complex Event Processing: Input-based and State-based Techniques. In *ICDE 2020.*